BRIEF

Architecture Feature

Guide interne de structuration et de maintenance

Conventions · Découpage · Hooks · Services · Workflow

Document de référence

Sommaire

1. Philosophie
2. Structure cible d'une feature
3. Conventions de nommage
4. Organisation interne des sous-dossiers
5. Hooks : data, UI, et le piège du fourre-tout
6. Le rôle du context
7. Types : organisation
8. Constants, theme, utils
9. Routes
10. Tests
11. Dépendances entre features
12. Quand créer une nouvelle feature ?
13. Checklist de revue d'architecture
14. Plan de migration de l'existant
15. Anti-patterns récurrents
16. Arbre de décision

1. Philosophie

1.1 Vertical Slice (Feature-First)

Le projet suit une architecture par feature (vertical slice), pas par type technique. Chaque feature est un dossier autonome regroupant tout ce qui la concerne : composants, hooks, types, constantes, thème, routes.

Conséquence : une feature doit pouvoir être lue, comprise et — en cas extrême — supprimée ou extraite sans toucher au reste du code.

1.2 Trois règles d'or

- **Co-localisation** : si deux fichiers évoluent ensemble, ils vivent ensemble.
- **Une seule manière de faire les choses** : si une convention existe, on s'y tient, partout.
- **Distance proportionnelle à la stabilité** : ce qui change ensemble est proche, ce qui est stable peut s'éloigner.

2. Structure cible d'une feature

Arborescence type d'une feature autonome :

```
📦 feature_name/
├ 📂 components/        # UI uniquement
│ ├ 📂 domain_a/        # un sous-dossier par domaine
│ ├ 📂 domain_b/
│ └ 📂 shared/          # composants réutilisés dans la feature
├ 📂 hooks/             # logique métier, état, side-effects
│ ├ 📂 context/         # providers React
│ └ 📜 useXxx.ts
├ 📂 services/          # accès données (Supabase, fetch, etc.)
├ 📂 routes/            # entry points
├ 📂 types/             # contrats TypeScript
├ 📂 utils/             # fonctions pures
├ 📜 constants.ts       # valeurs statiques
└ 📜 theme.ts           # tokens visuels propres à la feature
```

2.1 Rôle de chaque dossier

Dossier	Contient	Ne contient pas
components/	JSX, styling, props	Logique métier, fetch, state global
hooks/	Custom hooks (useXxx), state	Composants, types complexes
hooks/context/	Providers + consumer hooks	Logique métier dense
services/	Couche d'accès aux données	UI, state React
routes/	Composants-pages, entry points	Sous-composants partagés

Dossier	Contient	Ne contient pas
types/	Interfaces, types, enums	Logique runtime, validation
utils/	Fonctions pures (in → out)	Hooks, composants, fetch
constants.ts	Valeurs littérales, enums	Logique, dépendances externes
theme.ts	Tokens propres à la feature	Composants stylisés

3. Conventions de nommage

3.1 Règle générale

Élément	Convention	Exemple
Dossier de feature	snake_case ou kebab-case	stock_beta/, user-profile/
Sous-dossier de domaine	kebab-case	shipment/, delivery-form/
Composant React	PascalCase	ShipmentBoard.tsx
Hook custom	camelCase, préfixe use	useShipment.ts
Service / API	camelCase + suffixe	shipmentService.ts
Types	kebab-case ou domaine	shipment.types.ts
Constants	camelCase fichier, UPPER const	shipmentConstants.ts → MAX_ITEMS
Utils	camelCase	formatDate.ts, shipmentHelpers.ts
Test	suffixe .test	Shipment.test.tsx

3.2 Anti-patterns à bannir

```
❌ balleBoard.tsx          → camelCase pour un composant
✅ BalleBoard.tsx

❌ use_balle.tsx           → snake_case pour un hook
✅ useBalle.ts

❌ types/type_balle.ts     → préfixe redondant avec le dossier
✅ types/balle.types.ts

❌ helper_stock.ts         → snake_case isolé dans un projet camelCase
✅ stockHelpers.ts

❌ historyUtils.ts + activityHelpers.tsx + helper_stock.ts
   → trois conventions pour la même intention
✅ Choisir UN suffixe : .helpers.ts OU .utils.ts, partout
```

3.3 Langue : domaine vs technique

- **Domaine métier** : peut rester en français si l'équipe et le PO parlent métier en français (balle, fournisseur, livraison, arrivage).
- **Termes techniques** : toujours en anglais (board, form, sheet, filter, hook, service).
- **Interdit** : utiliser les deux pour la même entité (`Livraison/deliveryBoard.tsx` est un piège — choisir `Livraison/livraisonBoard.tsx` ou `Delivery/deliveryBoard.tsx`).

4. Organisation interne des sous-dossiers

4.1 Découpage par domaine (recommandé)

Tout ce qui touche à une entité métier vit dans un seul dossier, quelle que soit la nature UI.

```
components/
  └ shipment/
    ├ ShipmentBoard.tsx
    ├ ShipmentTableView.tsx
    ├ ShipmentCalendarView.tsx
    ├ ShipmentCreationSheet.tsx     ← formulaire ici, pas dans /Form
    ├ ShipmentActionMenu.tsx
    └ ShipmentFilters.tsx
```

Pourquoi pas un `components/forms/` transverse ? Parce que quand tu touches au formulaire de Shipment, tu touches aussi à son board, ses filtres, son type. Les avoir séparés t'oblige à naviguer en zig-zag. La cohésion par domaine bat la cohésion par « nature UI ».

4.2 Quand créer un sous-dossier ?

Situation	Action
< 4 fichiers liés à un domaine	À plat dans components/
4–10 fichiers	Sous-dossier components/{domain}/
> 10 fichiers ou plusieurs vues	Sous-sous-dossiers (views/, forms/, panels/) dans le domaine

4.3 Profondeur homogène

Si tu crées un sous-dossier pour une entité, **fais-le pour toutes les entités de même statut**. Ne pas avoir `shipment/` (sous-dossier) à côté de `balle.ts` (à plat) au même niveau hiérarchique. C'est l'erreur la plus fréquente.

5. Hooks : data, UI, et le piège du fourre-tout

5.1 Trois catégories de hooks, trois emplacements

```
hooks/
 ├ context/                  # Providers + hooks de consommation
 │ ├ ShipmentProvider.tsx
 │ └ useShipmentContext.ts
 ├ data/                     # Hooks qui orchestrent les services
 │ ├ useShipment.ts          # CRUD + cache
 │ └ usePacking.ts
 └ ui/                       # Hooks d'UX (modale, focus, raccourci)
   └ useShipmentFilters.ts
```

5.2 Règle : un hook ne fait pas d'appel réseau direct

```
// ❌ MAUVAIS — couche d'accès aux données mélangée au hook
// hooks/shipment/use_shipment.ts
export function useShipment() {
  const fetch = async () => {
    const { data } = await supabase.from('shipments').select('*');
    return data;
  };
  // ...
}

// ✅ BON — le service est isolé, le hook l'orchestre
// services/shipmentService.ts
export const shipmentService = {
  list: () => supabase.from('shipments').select('*'),
  create: (s: Shipment) => supabase.from('shipments').insert(s),
};

// hooks/data/useShipment.ts
export function useShipment() {
  const list = useCallback(() => shipmentService.list(), []);
  // ...
}
```

Pourquoi ? Le jour où tu changes Supabase pour autre chose (ou tu mockes pour les tests), tu n'as qu'**un** endroit à toucher.

6. Le rôle du context

Un context React doit être réservé à de l'état réellement partagé entre composants éloignés. Sinon, c'est du useState local ou un hook.

6.1 Critères pour créer un context

- Au moins **3 composants** non frères doivent y accéder.

- L'état change rarement, ou les consommateurs sont mémoïsés.
- L'alternative (props drilling) traverserait > 2 niveaux.

6.2 Pattern recommandé

```
// hooks/context/ShipmentContext.tsx
const ShipmentContext = createContext<ShipmentContextValue | null>(null);

export function ShipmentProvider({ children }: PropsWithChildren) {
  const value = useShipmentData(); // logique extraite dans un hook
  return <ShipmentContext.Provider
value={value}>{children}</ShipmentContext.Provider>;
}

export function useShipmentContext() {
  const ctx = useContext(ShipmentContext);
  if (!ctx) throw new Error('useShipmentContext must be used inside
ShipmentProvider');
  return ctx;
}
```

Important : la logique vit dans `useShipmentData`, pas dans le Provider. Le provider est un câblage, pas un cerveau.

7. Types : organisation

7.1 Un fichier par domaine

```
types/
 ├ shipment.ts          # tout ce qui concerne Shipment
 ├ delivery.ts
 ├ balle.ts
 └ shared.ts            # types transverses à la feature
```

7.2 Co-localisation des types triviaux

Un type utilisé par **un seul composant** vit avec le composant :

```
// components/shipment/ShipmentFilters.tsx
type FilterState = { status: 'all' | 'pending' | 'done' };

export function ShipmentFilters() { /* ... */ }
```

Pas la peine de polluer types/ pour ça.

7.3 Export des types

- Types **publics** de la feature : exportés depuis un index.ts à la racine de la feature.

- Types **internes** : pas exportés, ou exportés seulement depuis le dossier types/.

8. Constants, theme, utils

8.1 constants.ts

```
// stock/constants.ts
export const MAX_BALLE_WEIGHT = 50;
export const SHIPMENT_STATUS = ['draft', 'pending', 'done'] as const;
export type ShipmentStatus = typeof SHIPMENT_STATUS[number];
```

Si le fichier dépasse 100 lignes ou couvre plusieurs domaines, le splitter :

```
constants/
 ├ shipment.constants.ts
 └ balle.constants.ts
```

8.2 theme.ts

Réservé aux tokens visuels propres à la feature que tu ne veux pas dans le thème global. Si une couleur est utilisée hors de la feature, elle remonte dans le thème global.

8.3 utils/

Règles strictes pour qu'un fichier soit dans utils/ :

- Toutes les fonctions sont **pures** (mêmes entrées → mêmes sorties).
- Aucune dépendance à React (pas de hook, pas de JSX).
- Pas d'effet de bord (pas de console.log, pas de fetch).

Si une de ces règles n'est pas tenue, le code va dans hooks/ ou services/.

9. Routes

Le dossier routes/ contient les **entry points** : un fichier = une route déclarée dans le routeur.

```
routes/
 ├ Stock.tsx              # /stock
 ├ Stock.test.tsx
 └ StockSettings.tsx      # /stock/settings
```

Ces fichiers ne contiennent **presque rien** : ils orchestrent les providers et délèguent à des composants.

```
// routes/Stock.tsx
export default function Stock() {
  return (
    <ReferentialProvider>
```

```
      <StockProvider>
        <StockBoard />
      </StockProvider>
    </ReferentialProvider>
  );
}
```

10. Tests

- **Co-localisation** : Foo.test.tsx à côté de Foo.tsx.
- **Niveaux** : prioriser les tests d'intégration au niveau du routes/ et les tests unitaires sur les utils/ et hooks complexes.
- **Composants visuels** : laisser tomber les tests d'implémentation, préférer des tests de comportement (@testing-library/react).

11. Dépendances entre features

11.1 Règle : les features ne se parlent pas directement

```
❌ MAUVAIS
features/billing/components/Foo.tsx
  import { useStock } from '../../stock/hooks/useStock'

✅ BON
features/billing/components/Foo.tsx
  import { useStock } from '@/shared/stock'        // API publique exposée
```

11.2 Comment exposer une API publique

```
// stock/index.ts
export { useStock } from './hooks/data/useStock';
export type { Stock, Balle } from './types';
// Ne PAS ré-exporter les composants internes
```

Toute import qui plonge dans `stock/components/...` depuis une autre feature est un **smell** : soit le composant doit remonter dans shared/, soit la dépendance doit s'inverser (events, callbacks, props).

12. Quand créer une nouvelle feature ?

Critères cumulatifs :

- L'entité métier est nommée et comprise par le PO.
- Elle a sa propre route ou sa propre section UI principale.
- Elle a son propre cycle de vie (création / lecture / modification).
- Elle pèse au moins ~10 fichiers une fois implémentée.

Sinon : c'est un sous-domaine d'une feature existante.

13. Checklist de revue d'architecture

Avant de merger une feature, vérifier :

- Tous les composants sont en PascalCase.
- Tous les hooks commencent par use et sont en camelCase.
- Aucun fichier helper_xxx, xxxUtils et xxxHelpers ne cohabitent — une seule convention.
- Pas d'appel direct à supabase ou fetch dans un composant ou un hook : ça passe par services/.
- Pas de profondeur incohérente (a/b/c/d.ts à côté de a/e.ts au même niveau métier).
- Le routes/Xxx.tsx est court et ne fait que câbler.
- Aucun import inter-features ne plonge dans des chemins internes.
- Les types triviaux restent co-localisés ; seuls les types partagés sont dans types/.
- Les tests sont à côté de ce qu'ils testent.
- La langue (FR/EN) est cohérente pour chaque entité.

14. Plan de migration de l'existant

Pour la feature `stock` actuelle, voici l'ordre recommandé pour s'aligner sur cette doc sans tout casser :

- **1.** Renommer en masse les composants en PascalCase (un PR dédié, mécanique, faible risque).
- **2.** Renommer les hooks use_xxx.ts → useXxx.ts.
- **3.** Extraire la couche services : sortir supabase_arrivage.ts et tout fetch direct dans services/.
- **4.** Aplatir ou approfondir hooks/Stock/ pour avoir une profondeur homogène, et retirer la capitale.
- **5.** Fusionner components/stockboard/Form/ dans les dossiers de domaine respectifs (Balle/, Delivery/).
- **6.** Standardiser la langue par entité (choisir Delivery OU Livraison, pas les deux).
- **7.** Renommer les fichiers type_xxx.ts → xxx.types.ts, helper_xxx.ts → xxxHelpers.ts.
- **8.** Créer un index.ts à la racine de la feature pour exposer l'API publique.

Chaque étape = 1 PR, atomique, mergeable indépendamment.

15. Anti-patterns récurrents

À interdire en revue de code :

Anti-pattern	Symptôme	Correction
God-component	Un fichier > 400 lignes qui orchestre tout	Splitter par sous-responsabilité

Anti-pattern	Symptôme	Correction
Hooks de fetch déguisés	useXxx qui contient l'appel Supabase	Extraire dans services/, hook orchestre
Context fourre-tout	Un seul StockContext qui porte tout	Séparer par préoccupation (data, UI, settings)
Imports circulaires	A importe B qui importe A	Remonter le code commun dans utils/ ou shared/
Sous-dossier orphelin	Form/ qui ne contient qu'un fichier	Ramener à plat ou y mettre tout le domaine
Types dupliqués	Shipment défini deux fois	Source unique dans types/, ré-export ailleurs
Composant qui connaît Supabase	Composant qui appelle supabase.from()	Passer par un hook qui passe par un service

16. Arbre de décision : où je mets ce fichier ?

```
J'écris du code... est-ce que ça rend du JSX ?
├─ Oui  → est-ce une page entière liée à une route ?
│         ├─ Oui → routes/
│         └─ Non → components/{domain}/
└─ Non  → est-ce que ça appelle Supabase / un endpoint ?
          ├─ Oui → services/
          └─ Non → est-ce que ça utilise un hook React ?
                   ├─ Oui  → hooks/{context|data|ui}/
                   └─ Non  → est-ce une fonction pure ?
                             ├─ Oui → utils/
                             └─ Non → repenser la responsabilité du fichier
```

— Fin du document —